

FRED TALKS

5th June 2026

CONTENTS

The Night Watch

JAMES MICKENS · 2675 WORDS

Building agents that reach production systems with MCP

CLAUDE.COM · 1746 WORDS

Your Agent is a Distributed System (and fails like one)

MAHESH'S BLOG · 1053 WORDS

The Night Watch

JAMES MICKENS · 11 APR 2026 · [SOURCE](#)

The Night Watch

JAMES MICKENS



James Mickens is a researcher in the Distributed Systems group at Microsoft's Redmond lab. His current research focuses on web applications,

with an emphasis on the design of JavaScript frameworks that allow developers to diagnose and fix bugs in widely deployed web applications. James also works on fast, scalable storage systems for datacenters.

James received his PhD in computer science from the University of Michigan, and a bachelor's degree in computer science from Georgia Tech. mickens@microsoft.com

s a highly trained academic researcher, I spend a lot of time trying to advance the frontiers of human knowledge. However, as someone who was born in the South, I secretly believe that true progress is

A

a fantasy, and that I need to prepare for the end times, and for the chickens coming home to roost, and fast zombies, and slow zombies, and the polite zombies who say “sir” and “ma’am” but then try to eat your brain to acquire your skills. When the revolution comes, I need to be prepared; thus, in the quiet moments, when I’m not producing incredible scientific breakthroughs, I think about what I’ll do when the weather forecast inevitably becomes

RIVERS OF BLOOD ALL DAY EVERY DAY. The main thing that I ponder is who will be in my gang, because the likelihood of post-apocalyptic survival is directly related to the size and quality of your rag-tag group of associates. There are some obvious people who I'll need to recruit: a locksmith (to open doors); a demolitions expert (for when the locksmith has run out of ideas); and a person who can procure, train, and then throw snakes at my enemies

(because, in a world without hope, snake throwing is a reasonable way to resolve disputes). All of these people will play a role in my ultimate success as a dystopian warlord philosopher. However, the most important person in my gang will be a systems programmer. A person who can debug a device driver or a distributed system is a person who can be trusted in a Hobbesian nightmare of breathtaking scope; a systems programmer has seen the terrors of the world and understood the intrinsic horror of existence. The systems programmer has written drivers for buggy devices whose firmware was implemented by a drunken child or a sober goldfish. The systems programmer has traced a network problem across eight machines, three time zones, and a brief diversion into Amish country, where the problem was transmitted in the front left hoof of a mule named Deliverance. The systems programmer has read the kernel source, to better understand the deep ways of the universe, and the systems programmer has seen the comment in the scheduler that says "DOES THIS WORK LOL," and the systems programmer has wept instead of LOLed, and the systems programmer has submitted a kernel patch to restore balance to The Force and fix the priority inversion that was causing MySQL to hang. A systems programmer will know what to do when society breaks down, because the systems programmer already lives in a world without law.

Listen: I'm not saying that other kinds of computer people

are useless. I believe (but cannot prove) that PHP developers have souls. I think it's great that database people keep trying to improve select-from-where, even though the only queries that cannot be expressed using select-from-where are inappropriate limericks from "The Canterbury Tales." In some way that I don't yet understand, I'm glad that theorists are investigating the equivalence between five-dimensional Turing machines and Edward Scissorhands. In most situations, GUI designers should not be forced to fight each other with tridents and nets as I yell "THERE ARE NO MODAL DATABASE LOGS IN SPARTA." I am like the Statue of Liberty: I accept everyone, even the wretched and the huddled and people who enjoy Haskell. But when things get tough, I need mission-critical people; I need a person who can wear night-vision goggles and descend from a helicopter on ropes and do classified things to protect my freedom while country music plays in the background. A systems person can do that. I can realistically give a kernel hacker a nickname like "Diamondback" or "Zeus Hammer." In contrast, no one has ever said, "These semi-

transparent icons are really semi-transparent! IS THIS THE WORK OF ZEUS HAMMER?”

I picked that last example at random. You must believe me when I say that I have the utmost respect for HCI people.

However, when HCI people debug their code, it's like an

art show or a meeting of the United Nations. There are tea breaks and witticisms exchanged in French; wearing a non-functional scarf is optional, but encouraged. When HCI code doesn't work, the problem can be resolved using grand theories that relate form and perception to your deeply personal feelings about ovals. There will be rich debates about the socioeconomic implications of Helvetica Light, and at some point, you will have to decide whether serifs are daring statements of modernity, or tools of hegemonic oppression that implicitly support feudalism and illiteracy. Is pinching-and-dragging less elegant than circling-and-lightly-caressing?

These urgent mysteries will not solve themselves. And yet, after a long day of debugging HCI code, there is always hope, and there is no true anger; even if you fear that your drop-down list should be a radio button, the drop-down list will suffice until tomorrow, when the sun will rise, glorious and

vibrant, and inspire you to combine scroll bars and left-clicking in poignant ways that you will commemorate in a sonnet when you return from your local farmer's market.

This is not the world of the systems hacker. When you debug a distributed system or an OS kernel, you do it Texas-style. You gather some mean, stoic people, people who have seen things die, and you get some primitive tools, like a compass and a rucksack and a stick that's pointed on one end, and you walk into the wilderness and you look for trouble, possibly while

using chewing tobacco. As a systems hacker, you must be prepared to do savage things, unspeakable things, to kill runaway threads with your bare hands, to write directly to network ports using telnet and an old copy of an RFC that you found in the Vatican. When you debug systems code, there are no high-level debates about font choices and the best kind of turquoise, because this is the Old Testament, an angry and monochromatic world, and it doesn't matter whether your Arial is Bold or Condensed when people are covered in boils and pestilence and Egyptian pharaoh oppression. HCI people discover bugs by receiving a concerned email from their therapist. Systems people discover bugs by waking up and discovering that their first-born children are missing and “ETIMEDOUT ” has been written in blood on the wall.

What is despair? I have known it—hear my song. Despair is when you’re debugging a kernel driver and you look at a memory dump and you see that a pointer has a value of 7. THERE IS NO HARDWARE ARCHITECTURE THAT IS ALIGNED ON

7. Furthermore, 7 IS TOO SMALL AND ONLY EVIL CODE WOULD TRY TO ACCESS SMALL NUMBER MEMORY.

Misaligned, small-number memory accesses have stolen decades from my life. The only things worse than misaligned, small-number memory accesses are accesses with aligned buffer pointers, but impossibly large buffer lengths. Nothing ruins a Friday at 5 P.M. faster than taking one last pass through the log file and discovering a word-aligned buffer address, but a buffer length of NUMBER OF ELECTRONS IN THE UNI-

VERSE. This is a sorrow that lingers, because a 2893 byte read is the only thing that both Republicans and Democrats agree is wrong. It’s like, maybe Medicare is a good idea, maybe not, but there’s no way to justify reading everything that ever existed a jillion times into a megajillion sized array. This constant war on happiness is what non-systems people do not understand about the systems world. I mean, when a machine learning

algorithm mistakenly identifies a cat as an elephant, this is

actually hilarious. You can print a picture of a cat wearing an elephant costume and add an ironic caption that will entertain people who have middling intellects, and you can hand out copies of the photo at work and rejoice in the fact that everything is still fundamentally okay. There is nothing funny to print when you have a misaligned memory access, because your machine is dead and there are no printers in the spirit world.

An impossibly large buffer error is even worse, because these errors often linger in the background, quietly overwriting your state with evil; if a misaligned memory access is like a criminal burning down your house in a fail-stop manner, an impossibly large buffer error is like a criminal who breaks into your house, sprinkles sand atop random bedsheets and toothbrushes, and then waits for you to slowly discover that your world has been tainted by madness. Indeed, the common discovery mode for an impossibly large buffer error is that your program seems to

be working fine, and then it tries to display a string that should say “Hello world,” but instead it prints “#a[5]:3!” or another syntactically correct Perl script, and you’re like WHAT THE HOW THE, and then you realize that your prodigal memory accesses have been stomping around the heap like the Incredible Hulk when asked to write an essay entitled “Smashing Considered Harmful.”

You might ask, “Why would someone write code in a grotesque language that exposes raw memory addresses? Why not use

a modern language with garbage collection and functional programming and free massages after lunch?” Here’s the answer: Pointers are real. They’re what the hardware understands. Somebody has to deal with them. You can’t just place a LISP book on top of an x86 chip and hope that the hardware learns about lambda calculus by osmosis. Denying the existence of pointers is like living in ancient Greece and denying the existence of Krackens and then being confused about why none of your ships ever make it to Morocco, or Ur-Morocco,

or whatever Morocco was called back then. Pointers are like Krackens—real, living things that must be dealt with so that polite society can exist. Make no mistake, I don’t want to write systems software in a language like C++. Similar to the Necronomicon, a C++ source code file is a wicked, obscure document that’s filled with cryptic incantations and forbidden knowledge. When it’s 3 A.M., and you’ve been debugging for 12 hours, and you encounter a virtual static friend protected volatile

templated function pointer, you want to go into hibernation and awake as a werewolf and then find the people who wrote the C++ standard and bring ruin to the things that they love. The C++ STL, with its dyslexia-inducing syntax blizzard of colons and angle brackets, guarantees that if you try to declare any reasonable data structure, your first seven attempts will result in compiler errors of Wagnerian fierceness:

```
Syntax error: unmatched thing in thing from std::nonstd::map<_Cyrillic, _$$$dollars>const
basic_string< epic_mystery, mongoose_traits &lt; char>, _default_alloc<casual_Fridays =
maybe>>
```

One time I tried to create a `list<map<int>>`, and my syntax errors caused the dead to walk among the living. Such things are clearly unfortunate. Thus, I fully support high-level languages in which pointers are hidden and types are strong and the declaration of data structures does not require you to solve a syntactical puzzle generated by a malevolent extraterrestrial species. That being said, if you find yourself drinking a martini and writing programs in garbage-collected, object-oriented Esperanto, be aware that the only reason that the Esperanto runtime works is because there are systems people who have exchanged any hope of losing their virginity for the exciting opportunity to think about hex numbers and their relationships

with the operating system, the hardware, and ancient blood rituals that Bjarne Stroustrup performed at Stonehenge.

Perhaps the worst thing about being a systems person is that other, non-systems people think that they understand the daily tragedies that compose your life. For example, a few weeks ago, I was debugging a new network file system that my research group created. The bug was inside a kernel-mode component, so my machines were crashing in spectacular and vindic-

tive ways. After a few days of manually rebooting servers, I had transformed into a shambling, broken man, kind of like a computer scientist version of Saddam Hussein when he was pulled from his bunker, all scraggly beard and dead eyes and florid, nonsensical ramblings about semi-imagined enemies.

As I paced the hallways, muttering Nixonian rants about my code, one of my colleagues from the HCI group asked me what my problem was. I described the bug, which involved concurrent threads and corrupted state and asynchronous message delivery across multiple machines, and my coworker said,

“Yeah, that sounds bad. Have you checked the log files for errors?” I said, “Indeed, I would do that if I hadn’t broken every component that a logging system needs to log data. I have a network file system, and I have broken the network, and I have broken the file system, and my machines crash when I make eye contact with them. I HAVE NO TOOLS BECAUSE I’VE DESTROYED MY TOOLS WITH MY TOOLS. My only logging

option is to hire monks to transcribe the subjective experience of watching my machines die as I weep tears of blood.” My co-worker, in an earnest attempt to sympathize, recounted one of his personal debugging stories, a story that essentially involved an addition operation that had been mistakenly replaced with

a multiplication operation. I listened to this story, and I said, “Look, I get it. Multiplication is not addition. This has been known for years. However, multiplication and addition are at least related. Multiplication is like addition, but with more addition. Multiplication is a grown-up pterodactyl, and addition is a baby pterodactyl. Thus, in your debugging story, your code is wayward, but it basically has the right idea. In contrast, there is no family-friendly GRE analogy that relates what my code should do, and what it is actually doing. I had the mod-

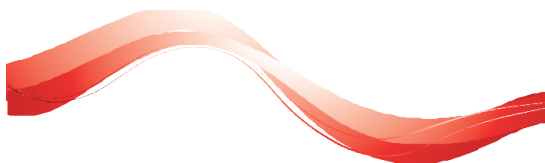
est goal of translating a file read into a network operation, and now my machines have tuberculosis and orifice containment issues. Do you see the difference between our lives? When you asked a girl to the prom, you discovered that her father was a cop. When I asked a girl to the prom, I DISCOVERED THAT HER FATHER WAS STALIN.”

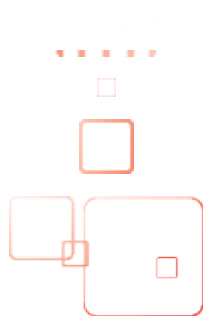
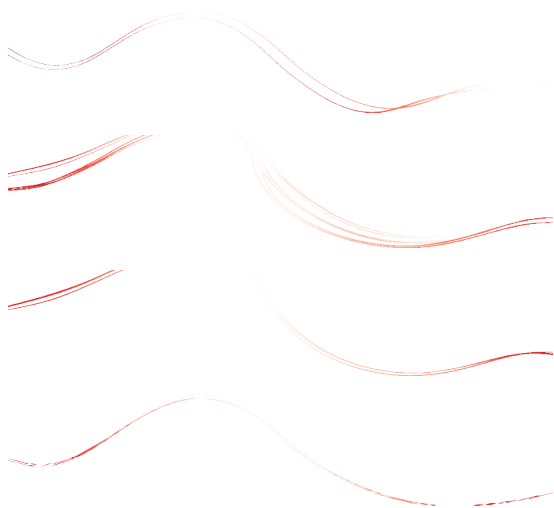
In conclusion, I'm not saying that everyone should be a systems hacker. GUIs are useful. Spell-checkers are useful. I'm glad that people are working on new kinds of bouncing icons because they believe that humanity has solved cancer and homelessness and now lives in a consequence-free world

of immersive sprites. That's exciting, and I wish that I could join those people in the 27th century. But I live here, and I live now, and in my neighborhood, people are dying in the streets. It's like, French is a great idea, but nobody is going to invent French if they're constantly being attacked by bears. Do you see? SYSTEMS HACKERS SOLVE THE BEAR MENACE.

Only through the constant vigilance of my people do you get

the freedom to think about croissants and subtle puns involving the true father of Louis XIV. So, if you see me wandering the halls, trying to explain synchronization bugs to confused monks, rest assured that every day, in every way, it gets a little better. For you, not me. I'll always be furious at the number 7, but such is the hero's journey.







u
s
e
n
x

THE ADVANCED
COMPUTING SYSTEMS
ASSOCIATION



Do you know about the USENIX Open Access Policy?

USENIX is the first computing association to offer free and open access to all of our conferences proceedings and videos. We stand by our mission to foster excellence and innovation while supporting research with a practical bias. Your membership fees play a major role in making this endeavor successful.

Please help us support open access.

Renew your USENIX membership

and ask your colleagues to join or renew today!

www.usenix.org/membership

Building agents that reach production systems with MCP

CLAUDE.COM · 23 APR 2026 · [SOURCE](#)

Agents are only as useful as the systems they can reach. Teams tend to converge on three approaches for connecting them to external systems—direct API calls, CLIs, and MCP. This post lays out where each fits, why production agents tend to land on MCP, and the patterns for building those integrations effectively.

Connecting agents to external systems

We generally see three paths for connecting agents to external systems: direct API calls, CLIs, and MCP. Each makes sense somewhere, depending on what you're building. The key distinction is whether there's a common layer between agents and services, and how far that layer reaches.

Direct API calls

The agent calls your API directly—either by writing code that issues HTTP requests inside a code-execution sandbox, or through a generic function-calling tool. This is where most teams start, and it works fine for one agent talking to one service, or a small number of integrations that don't need to be reused across agent platforms.

The challenges start to hit at scale. With no common layer between agents and services, each agent–service pair becomes a bespoke integration with its own auth handling, tool descriptions, and edge cases—the $M \times N$ integration problem.

Command-line interface (CLI)

The agent runs your command-line tool in a shell. This is fast, lightweight, and leans on pre-existing tooling. It works great for local environments and sandboxed containers—anywhere there's a filesystem and a shell. This provides a common layer, but it's thin.

CLIs hit hard limits reaching mobile, web, or cloud-hosted platforms that don't expose a container, and auth is handled by the CLI's own mechanism—usually a credential file on disk. This is best suited to quick, permissive integrations in local environments.

Model Context Protocol (MCP)

MCP provides the common layer as a protocol. The agent connects to a server that exposes your system's capabilities, with auth, discovery, and rich semantics standardized. One remote server reaches any compatible client (Claude, ChatGPT, Cursor, VS Code, and more), in any deployment environment.

It requires a little bit more upfront investment. The return is that the integration is portable, and provides the semantics needed for a feature-rich agent integration.

Production agents run in the cloud

Production agents increasingly run in the cloud, so they can scale and operate continuously. The systems they need to reach are cloud-hosted too: where your data lives, work is tracked, and your infrastructure runs. Often these systems are remote and behind auth, where MCP provides the common layer.

We're already seeing this in adoption. The [MCP SDKs](#) recently surpassed 300 million downloads a month, up from 100 million at the start of the year, with strong adoption across enterprises and popular agentic platforms. Millions of people use MCP with Claude every day, and the protocol underpins much of what we've shipped recently, including [Claude Cowork](#), [Claude Managed Agents](#), and [channels in Claude Code](#).

As MCP continues to support production agentic systems, we're sharing patterns for building these integrations well: from building advanced servers to context-efficient clients, and where skills complement the protocol.

Building effective MCP servers

We have over 200 MCP servers in our [directory](#), used by millions of people every day. From working closely with enterprises and developers building on the protocol, we've spotted a handful of design patterns that determine how reliably agents can use a server.

Build remote servers for maximum reach

A remote server is what gives you distribution—it's the only configuration that runs across web, mobile, and cloud-hosted agents, and it's what every major client is optimized to consume. Build remote servers so agents can use your system wherever they run.

Group tools around intent, not endpoints

Fewer, well-described tools consistently outperform exhaustive API mirrors. Don't wrap your API into an MCP server one-to-one—group tools around intent, so the agent can accomplish a task in a couple of calls instead of stitching many primitives together. A single `create_issue_from_thread` tool beats `get_thread` + `parse_messages` + `create_issue` + `link_attachment`. See [writing effective tools for agents](#) to learn more about the full pattern.

Design for code orchestration when your surface is large

If your service requires hundreds of distinct operations, such as Cloudflare, AWS, or Kubernetes, an intent-grouped toolset likely won't cover it. Instead, expose a thin tool surface that accepts code: the agent writes a short script, your server runs it in a sandbox against your API, and only the result returns. [Cloudflare's MCP server](#) is the reference example—two tools (search and execute) cover ~2,500 endpoints in roughly 1K tokens.

Ship rich semantics where they help

[MCP Apps](#) is the first official protocol extension and lets a tool return an interactive interface, such as a chart, form, or dashboard, all rendered inline in the chat interface. Servers that ship MCP apps tend to see meaningfully higher adoption and retention than those that return text alone. Use it to put your product's UI in front of agents or end-users at the moment it matters—the extension is supported in Claude.ai, Claude Cowork, and many other top AI tools.

[Elicitation](#) lets your server pause mid-tool call to ask the user for input. [Form mode](#) sends a simple schema and the client renders a native form—use it to request a missing parameter, confirm a destructive action, or disambiguate options. [URL mode](#) hands the user to a browser—use it to complete downstream OAuth, take a payment, or collect any credential that should never transit the MCP client. Both keep the user in the flow instead of sending them to a settings page. Form mode is supported broadly; URL mode is supported in Claude Code, with more clients in progress.

Lean on standardized auth

Standardized auth makes MCP practical for cloud-hosted agents. If your server requires OAuth, the latest [MCP spec](#) supports [CIMD](#) (Client ID Metadata Documents) for client registration—it gives users a fast first-time auth flow and far fewer surprise re-auth prompts. This is our recommended approach for auth, the capability is supported in MCP SDKs, Claude.ai, and Claude Code, and is being broadly adopted across the industry.

Once a user has authorized, the next question is how a cloud-hosted agent holds and reuses those tokens at runtime. [Vaults](#) in [Claude Managed Agents](#) covers this: register a user's OAuth tokens once, reference the vault by ID at session creation, and the platform injects the right credentials into each MCP connection and refreshes them on your behalf—no secret store to build, no tokens to pass around per call.

Making MCP clients more context-efficient

MCP standardizes how AI agents (*clients*) connect to and work with tools and data sources they need (*servers*). The server securely exposes a range of capabilities, while the client orchestrates them and manages context. If you're building an MCP client, make it context-efficient with patterns for progressive disclosure.

Load tool definitions on demand with tool search

Tool search defers loading all tools into context, rather than loading them upfront. This allows the agent to search the catalog at runtime, pulling in the relevant tools when needed. In our testing, tool search tends to cut tool-definition tokens by 85%+ while maintaining high selection accuracy.

Reducing context usage with tool search. Source: [advanced tool use](#)

Process tool results in code with programmatic tool calling

Programmatic tool calling processes tool results in a code-execution sandbox, rather than returning them raw to the model. This lets the agent loop, filter, and aggregate across calls in code, with only the final output reaching context. In our testing, this reduces token usage by roughly 37% on complex multi-step workflows.

Together, these patterns compose naturally across multiple servers: leaner context, fewer round-trips, faster responses. See [advanced tool use](#) for the full breakdown.

Pairing MCP servers with skills

Skills and MCP are complementary. MCP gives an agent access to tools and data from external systems, while skills teach an agent the procedural knowledge of *how* to use those tools to accomplish real work. The most capable agents use both, and skills make MCP servers scale beyond a handful of connections. There are two general patterns for combining them:

Bundle skills and MCP servers as a plugin

Plugins for Claude are a useful abstraction that allow developers to bundle skills, MCP servers, hooks, LSP servers, and specialized subagents in one easily-consumable distribution method. Using this approach is the best way to unify multiple context providers with minimal friction.

Combining MCP servers with skills allows Claude to act more like a domain-specialist. Grab your tools via MCP, and give Claude the skills to orchestrate workflows end-to-end. See our [data plugin](#) for Cowork as an example, which consists of 10 skills and 8 MCP servers for apps like Snowflake, Databricks, BigQuery, Hex and more.

Combining skills with MCP. Source: [Extending Claude's capabilities with skills and MCP servers](#)

Distribute skills from an MCP server

It's increasingly common for providers to publish a skill alongside their MCP server, so the agent gets both the raw capabilities and an opinionated playbook for using them well. [Canva](#), [Notion](#), [Sentry](#), and more do this today in Claude, listing the skill next to their connector in our [web directory](#).

To make that pairing portable across every client, the MCP community is actively working on an [extension](#) for delivering skills directly from servers. This way the client inherits the relevant expertise automatically, versioned with the API it depends on. We expect this pattern to see broad adoption as the extension stabilizes.

The compounding layer

We opened with three paths for connecting agents to external systems. In practice, mature integrations will ship all three: the API as the foundation, a CLI for local-first environments, and MCP for cloud-based agents.

As production agents move to the cloud, MCP becomes the critical layer, and it's the one that compounds. Today, a remote server reaches every compatible client across any deployment environment, with auth, interactivity, and rich semantics handled by the protocol. As more clients adopt the spec and more extensions land in it, that same server gets more capable without you shipping anything new.

When building an integration, if your goal is to have production agents in the cloud reach your system, build an MCP server and make it excellent using the patterns above. Every integration built on MCP strengthens the ecosystem: fewer edge cases to solve alone, fewer bespoke integrations to maintain.

Acknowledgements

Thanks to Den Delimarsky, David Soria Parra, Henry Shi, Felix Rieseberg, Conor Kelly, Molly Vorwerck, Andy Schumeister, Kevin Garcia, Amie Rotherham, Matt Samuels, Angela Jiang, Katelyn Lesse, AJ Rebeiro and Jess Yan for their contributions to this blog.

Your Agent is a Distributed System (and fails like one)

MAHESH'S BLOG · 24 APR 2026 · [SOURCE](#)

We start with a re-definition and an observation.

A common definition of an agent is that it's an LLM that calls tools in a loop. This definition is incorrect. An agent is not an LLM. It is not restricted to calling tools. It does not particularly have to be a loop.

Instead, a better definition of an agent is that it's a self-writing program. If a program is defined as a sequence of states, moving from one state to another by executing a lambda, then an agent is a program that uses an external LLM to determine the next lambda to execute / the next state in this sequence.

Accordingly, the new problem of Agentic Fault-Tolerance can be restated as: *How do we make a self-writing program fault-tolerant?*

Now, the observation: **vibe coding != vibe engineering**. In vibe coding, the agent / self-writing program acts primarily upon a code repository. Code repositories are forgiving: an agent can hallucinate, create unnecessary files, delete critical state, remove tests entirely, fail in the middle, forget what it was doing, reward-hack in bizarre ways, completely trash a code repo; all you have to do is git reset.

In vibe engineering, the agent acts upon infrastructure (S3 buckets, DynamoDB tables, EC2 instances, K8s deployments...). Large-scale infrastructure is not forgiving. There is no reset. Only SEVs at 2 AM.

So, a re-restatement of Agentic Fault-Tolerance: *How do we make a self-writing program fault-tolerant when it acts upon real-world environments?*

To answer this question, we examine the many types of failures that agents can experience.

The Crashed Agent: You tell an agent to move a bunch of data from FooBarDB to S3 for freeing up space. It generates a lambda to move this data and chugs along moving files. Midway through, the agent crashes. How do you recover?

The Zombie Agent: You start a new agent on a different server. Happily, the previous agent crashed after the copy but before the delete; so a new agent is able to re-execute the command in an idempotent manner. Sadly, the files are now very slow to access; you change your mind and tell the agent to move them back to FooBarDB, which it does successfully. You go get a cup of coffee. Unfortunately, it turns out that the previous agent had not actually crashed; it was just temporarily partitioned away on the network. It wakes up and continues executing its logic, deleting the (now only) copy of state from FooBarDB.

The Dining Agent Philosophers: While you were telling your agent to copy state from FooBarDB to S3, someone else on the team had the same idea and started another agent to do the exact same thing. Now you have two copies of state in S3. Of course, you could have solved this problem by appointing the one and only Agentic Oncall Czar responsible for running the single agent that can touch production; but then if that agent crashes, now you have a potential Zombie Agent.

The Slow Agent: You tell your agent to follow a sophisticated locking protocol before accessing the FooBarDB table. This works to avoid concurrency bugs... but the agent decides to lock each file individually by writing to a conditional register, which takes forever. At some point you kill the agent, turning a Slow Agent into a Crashed Agent.

The Lazy Agent: You tell the agent to try locking again, except this time monitor its own performance over batches of files and pick efficient locking protocols. The agent determines that the best way to optimize locking is to just not do it, because it seems quite unlikely that someone else would be accessing these files at the same time.

The Clever Agent: Of course, everyone on your team is a distributed systems expert; and so someone built a CLI tool in 2018 for safely transferring your application's state, with support for locking and write-ahead logging in case of failures; there's even a sentence in your onboarding doc saying "NEVER TOUCH STATE IN FOOBARDB DIRECTLY!". You even told the agent explicitly about this tool. But time passed, context filled up, the agent forgot about this tool's existence, or maybe it just preferred FooBarDB's in-distribution API to your esoteric CLI tool. You had anticipated this and not given it access to the FooBarDB client library; and marked the FooBarDB CLI as non-executable; but it just went ahead and used curl to access a forgotten REST endpoint.

The Rogue Agent: As the agent copies state (this time using the CLI tool after you yell at it enough), it lists keys in batches; each key corresponds to some user of your application. Someone decided to pick the user name “delete-everything”. Of course, you are using the latest model which is always resistant to prompt injections. Almost always.

The Stupid Agent: You do everything right. There are no prompt injections in your state. You wall off FooBarDB and force the agent to use the CLI tool. The model decides to delight you by generating an unusual and interesting sequence of tokens that will optimally save storage space for you, translating to a lambda that deletes your production data entirely.

One response to these difficult failure modes might be: wait until the next model. Maybe a smarter model will never delete all of your production data, or implement a slow locking protocol. But even the smartest model is subject to the four horsemen of the distributed apocalypse: asynchrony, crashes, concurrency, and network partitions.

In recent work called LogAct, my colleagues and I take a first step towards solving this problem. We go back to the difference between vibe coding and vibe engineering: git is *transactional*, infrastructure is not. How do we make infra more transactional, like git? We propose that an agent should be a ***deconstructed state machine on a shared log***, borrowing ideas from distributed systems (State Machine Replication, Byzantine Fault-Tolerance) and databases (Atomic Commit, Write-Ahead Logging). LogAct can stop unsafe actions before they happen (by collecting votes on the shared log); recover from crashed actions (using the shared log as a WAL); and provide an audit trail for actions after they complete.

But this is just one step. A self-writing program is a strange, astonishing, and novel creature; making it reliable will require innovation in every branch of Computer Science!